Lecture 13: Unsupervised Learning: Clustering and Mixture Models

Readings: ISL (Ch. 12), PRML[1] (Ch. 9), ESL (Ch. 14); code

Soon Hoe Lim

October 7, 2025

[1] In addition to ISL and ESL, we use Bishop's PRML as our reference for unsupervised learning, which provides more in-depth treatment of core methods with a unified probabilistic framework. ESL Ch. 14 covers a broader range of topics but with less depth on the fundamentals we emphasize.

Outline

1. Unsupervised Learning: Overview
2. Clustering
3. K-Means Clustering
4. Gaussian Mixture Models
5. Exercises

Unsupervised Learning: Overview

Given dataset $\{x_i\}_{i=1}^N$ without labels, where $x_i \in \mathbb{R}^d$ (equivalently, given data matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$), the main goal is to discover structure or learn properties of the underlying data distribution.

Key differences from supervised learning:

- Not interested in prediction: no labels/responses to guide learning
- More subjective - no simple goal for the analysis
- Success depends on interpretability and downstream utility
- Often used for exploratory data analysis: visualization and preprocessing

Main tasks in unsupervised learning (we focus on the first one today):

1. **Clustering**: Partition data into groups with similar characteristics
2. **Dimensionality Reduction**: Find low-dimensional representation that retains key information
3. **Density Estimation**: Model distribution from which data are sampled
4. **Generative Modeling**: Build models that can produce new samples from learned distribution

Clustering: Overview

Definition 1: Clustering

Partition N observations into K groups (clusters) such that observations within each cluster are more similar to each other than to observations in other clusters.

Requires specification of:

- **Dissimilarity measure** $d(x_i, x_{i'})$ between observations
- **Clustering algorithm** defining how dissimilarities are used
- **Number of clusters** K (often unknown)

Common dissimilarity measures:

- Euclidean (ℓ_2) distance: $d(x_i, x_{i'}) = \|x_i - x_{i'}\|_2$
- Manhattan (ℓ_1): $d(x_i, x_{i'}) = \|x_i - x_{i'}\|_1$
- Correlation-based: $d(x_i, x_{i'}) = 1 - \text{corr}(x_i, x_{i'})$

There are many different clustering methods. We focus on K-means clustering and Gaussian mixture models (see ESL Ch. 14 for other clustering algorithms).

K-Means Clustering: Intuitive Formulation

Definition 2: K-Means Clustering (Intuitive Definition)

Given N observations $\{x_i\}_{i=1}^N$ with $x_i \in \mathbb{R}^d$, the goal of **K-means clustering** is to partition the data into K distinct, non-overlapping clusters $C_1, \ldots, C_K$ that minimize the total within-cluster variation: $W(C) = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - m_k\|_2^2$, where $m_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i$ is the mean (centroid) of cluster k.

Each cluster has a "center"; assignments minimize within-cluster variance.

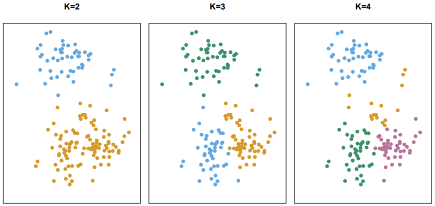


FIGURE 12.7. *A simulated data set with 150 observations in two-dimensional space. Panels show the results of applying K-means clustering with different values of K, the number of clusters. The color of each observation indicates the cluster to which it was assigned using the K-means clustering algorithm. Note that there is no ordering of the clusters, so the cluster coloring is arbitrary. These cluster labels were not used in clustering; instead, they are the outputs of the clustering procedure.*

K-Means Clustering

This is a difficult problem to solve (there are almost K^N ways to partition N observations into K clusters). But a simple algorithm (K-Means) can be shown to provide a local optimum to the optimization problem.

Let's introduce some notation:

- **Cluster representatives (centers):** $M = [m_1 | \dots | m_K] \in \mathbb{R}^{d \times K}$ (this is the matrix with columns consisting of the cluster centers $m_k \in \mathbb{R}^d$).
- **Assignment variables**

$$r_{ik} = \begin{cases} 1, & \text{if } x_i \text{ is assigned to cluster } k, \\ 0, & \text{otherwise,} \end{cases} \quad \text{with} \ \sum_{k=1}^{K} r_{ik} = 1.$$

The matrix $R = (r_{ik}) \in \{0,1\}^{N \times K}$ encodes cluster assignments (**1-of-K coding scheme**).

*K-means derives its name from the fact that each cluster center m_k is the mean of the observations assigned to that cluster.

K-Means: Matrix Formulation

Then the K-means **objective (distortion function)** is:

$$J(R, M) = \sum_{i=1}^{N} \sum_{k=1}^{K} r_{ik} \|x_i - m_k\|_2^2,$$

where $M \in \mathbb{R}^{d \times K}$ and $R \in \{0, 1\}^{N \times K} \subset \mathbb{R}^{N \times K}$.

Goal: We seek to minimize $J(R, M)$ **jointly** over both assignments and cluster centers:

$$\min_{R,M} J(R, M) \quad \text{s.t. } r_{ik} \in \{0, 1\}, \; \sum_{k=1}^{K} r_{ik} = 1.$$

K-Means: Coordinate Descent Interpretation

The optimization of $J(R, M)$ is intractable globally, but can be minimized iteratively by alternating two steps.

(1) Assignment step (update R): Suppose we know M, then R is determined by assigning each x_i to the nearest m_k:

$$r_{ik} = \begin{cases} 1, & \text{if } k = \arg\min_j \|x_i - m_j\|_2^2, \\ 0, & \text{otherwise.} \end{cases}$$

Ties are broken arbitrarily (e.g., take k to be the smallest index value of the minimizing indices).

(2) Update step (update M): Given fixed assignments R, minimize $J(R, M)$ with respect to M:

$$\nabla_{m_k} J(R, M) = 2 \sum_{i=1}^{N} r_{ik}(m_k - x_i) = 0 \quad \Rightarrow \quad m_k = \frac{\sum_{i=1}^{N} r_{ik} x_i}{\sum_{i=1}^{N} r_{ik}}.$$

Each cluster center m_k is the mean of all data points assigned to it.

K-Means Clustering (Lloyd's Algorithm)

K-Means (Lloyd's Algorithm)

1: **Input:** Data $\{x_i\}_{i=1}^N$, number of clusters K, stopping criterion
2: **Initialize:** Choose initial centers $M \in \mathbb{R}^{K\times d}$ (e.g., random data points)
3: **repeat**
4: **Assignment step:** For each $i = 1, \ldots, N$,

$$r_{ik} = \begin{cases} 1, & \text{if } k = \arg\min_j \|x_i - m_j\|_2^2, \\ 0, & \text{otherwise.} \end{cases}$$

5: **Update step:** For each $k = 1, \ldots, K$, $m_k = \frac{\sum_{i=1}^N r_{ik} x_i}{\sum_{i=1}^N r_{ik}}$.
6: **until** stopping criterion is reached (e.g., convergence of M or $J(R, M)$)
7: **return** centers M, assignments R

Complexity: $O(NKdT)$ where T is the number of iterations (typically small).

Connecting the Two Formulations

The intuitive and matrix formulations of K-means are equivalent.

Cluster set representation:

$$C_k = \{\, i : r_{ik} = 1 \,\} \subseteq \{1, \ldots, N\}.$$

Then

$$m_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i, \quad J(R, M) = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - m_k\|_2^2.$$

Summary:

- Intuitive view (look at the sets C_k): groups of similar points around a mean.
- Matrix view (R, M): binary assignment matrix and representative vectors.
- Both lead to the same objective and Lloyd's iterative algorithm.

K-Means: Properties and Initialization

Properties:

- Converges to local minimum. In fact, one can show that it must converge after a finite number of iterations (see Exercise 13.2).
- Not guaranteed to find global minimum (why?)
- Sensitive to initialization

Initialization strategies:

1. Random: Select K data points randomly as initial centers
2. K-means++: Choose first center randomly, then each subsequent center with probability proportional to squared distance from nearest existing center
3. Multiple random restarts: Run K-means multiple times with different random initializations, keep best result

Choosing K for K-Means

- **Elbow method**: Plot within-cluster sum of squares $W(C) = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - m_k\|^2$ vs K. As K increases, $W(C)$ always decreases. Choose K at the "elbow" where the rate of decrease slows significantly. Simple but subjective; elbow may not be clear.
- **Gap statistic**: Compare $\log W(C)$ for the data to $\mathbb{E}[\log W(C)_{\text{ref}}]$ computed from uniform random reference data. Choose K where the gap is largest. More principled than elbow method but computationally expensive (requires multiple bootstrap samples).
- **Silhouette analysis**: For each point i in cluster C_k, compute $s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1]$, where a_i is the average distance to other points in C_k and b_i is the minimum average distance to points in any other cluster. Choose K that maximizes the average silhouette score across all points.
- **Domain knowledge**: Often the most reliable approach. Use subject-matter expertise to determine a meaningful number of groups for your application.

⚠ No universally best method exists. In practice, try multiple values of K and validation approaches, and compare results for consistency and interpretability.

Hierarchical Clustering

Unlike K-means, in hierarchical clustering it's not required to commit to a particular choice of K. Instead, we produce a hierarchy of nested clusters, visualized as a **dendrogram** (tree-like visual representation of the observations), that allows us to view at once the clusterings obtained for dendrogram each possible number of clusters. See ISL Ch. 12.4.2 for details and visualizations.

On the choice of dissimilarity measure:

- **Euclidean distance**[2]**:** compares magnitudes.
- **Correlation distance:** compares shapes (patterns).

In general, scaling matters for clustering methods! Variables with larger variance can dominate. Should the observations or features first be standardized in some way? For standardized data, squared Euclidean and correlation distances can be shown to be equivalent (see Exercise 13.1).

💡 The choice of dissimilarity measure and scaling affects clustering results. Choose distance and scaling based on what "similarity" means for your data.

[2] Here we only consider squared Euclidan distance, but one can extend this to arbitrary dissimilarity measure with the K-medoids (see ESL Ch. 14.3.10).

Practical Issues in Clustering

- **Small decisions with major impact:** standardization, dissimilarity measure, number of clusters
- **Validation challenge:** clusters may reflect noise rather than true subgroups; no consensus on best validation approach[3] (see ESL Ch. 14)
- **Sensitive to data perturbations:** generally not very robust
- **Standardization:** raw vs. scaled variables can dramatically change results
- **Best practice:** try multiple parameter choices, cluster data subsets, look for consistent patterns across approaches
- **Interpretation:** results are exploratory starting points, not absolute truth—use to generate hypotheses for further study

⚠ Since all observations forced into clusters, the clusters found may be heavily distorted due to the presence of outliers. Mixture models are an attractive approach for accommodating the presence of such outliers.

[3] Common validation metrics include: silhouette score (measures cluster cohesion vs. separation), adjusted Rand index (compares clusterings when true labels available), within-cluster sum of squares (internal cohesion measure), and gap statistic (compares clustering to random data).

Gaussian Mixture Models (GMMs)

Probabilistic approach[4]: A soft alternative to K-means. Instead of a hard assignment to one cluster, GMMs[5] assign each point x_i a probability distribution over all K clusters (responsibilities), making them better equipped to handle uncertainty and outliers.

Definition 3: Gaussian Mixture Model (GMM)

Model data as coming from mixture of K Gaussians: $p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x; \mu_k, \Sigma_k)$ where $\pi_k \geq 0$, $\sum_{k=1}^{K} \pi_k = 1$ (mixture weights), and:

- $\mu_k \in \mathbb{R}^d$ (cluster means)
- $\Sigma_k \in \mathbb{R}^{d \times d}$ (positive definite covariance matrices)
- $\mathcal{N}(x; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)\right)$ (Gaussian PDF)

GMMs are more flexible than K-means as the covariance matrices Σ_k allow for elliptical clusters of varying size and orientation.

[4] The idea is to model the data as samples from some unknown probability distribution with K clusters, and our goal is to approximate this distribution. Here, use the GMM to parameterize this distribution.

[5] We have seen GMMs as a density estimation method in Lecture 5.

GMM: Latent Variable Interpretation

- For each data point x_i, we assume a latent variable $z_i \in \{1, \ldots, K\}$ indicates which component generated it.
- The **prior probability** of selecting component k is $p(z_i = k) = \pi_k$. The state $z_i = k$ can be represented by a one-hot vector $r_i \in \{0, 1\}^K$, with $\sum_{k=1}^K r_{ik} = 1$. The probability distribution over these vectors is $p(r_i) = \prod_{j=1}^K \pi_j^{r_{ij}}$.
- The **conditional density** of observing x_i given it came from component k is $p(x_i | z_i = k) = \mathcal{N}(x_i; \mu_k, \Sigma_k)$.
- Using the one-hot vector, this is written as $p(x_i | r_i) = \prod_{j=1}^K \mathcal{N}(x_i; \mu_j, \Sigma_j)^{r_{ij}}$.
- The **marginal density** of x_i is recovered by summing over all possible latent states, which yields the GMM formula:

$$p(x_i) = \sum_{k=1}^{K} p(z_i = k) p(x_i | z_i = k) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k).$$

*Throughout, we use lowercase $p(\cdot)$ for both probability densities (continuous variables like X so that $p(x) = \mathbb{P}(X = x)$) and probability mass functions (discrete variables like Z so that $p(Z = k) = \mathbb{P}(Z = k)$), as well as for the conditional versions. This notation is also used in PRML Ch. 9.

GMM: Posterior Probability (Responsibility)

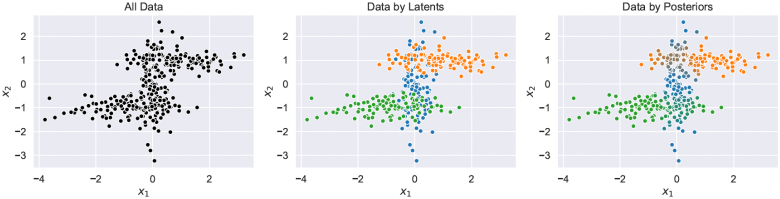
Then, using Bayes' rule, we find the **posterior probability** after observing x_i. This is the "responsibility" of component k for explaining x_i:

$$\gamma_{ik} := p(z_i = k|x_i) = \frac{p(x_i|z_i = k)p(z_i = k)}{p(x_i)} = \frac{\pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}.$$

γ_{ik} is precisely the expected value of the latent indicator variable r_{ik} (which is a Bernoulli random variable conditioned on x_i). Contrast this with the r_{ik} in K-Means where $r_{ik} \in \{0, 1\}$ (hard assignment).

Example:

We consider a 2D dataset generated from a mixture of three Gaussians shown in the left figure below. In the middle figure, we mark the dataset according to which Gaussian the data is generated from in the underlying latent generation process $\{r_{ik}\}$. In the far right figure, we plot the same dataset, but with colors representing the posteriors computed via (3.63), which shows which cluster explains each data point. We can see that in the overlaps of the 3 Gaussians, the posteriors take intermediate values.



GMM: Expectation-Maximization (EM) Algorithm

Goal: Find parameters $\theta = \{\pi_k, \mu_k, \Sigma_k\}$ that maximize the log-likelihood for i.i.d. samples $\{x_i\}_{i=1}^N$:

$$\log p(\mathbf{X}|\theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k) \right).$$

Challenge: We cannot directly maximize this because the cluster assignments (latent variables z_i) are unknown. Moreover, the optimal parameters and responsibilities are interdependent: to compute γ_{ik} we need θ, and to compute θ we need $\{\gamma_{ik}\}$.

EM Solution: Break optimization into two alternating steps:

- **E-step (Expectation)**: Given current parameters θ^{old}, compute what we *expect* the cluster assignments to be. Calculate responsibilities $\gamma_{ik} = p(z_i = k|x_i, \theta^{\text{old}})$.
- **M-step (Maximization)**: Given these expected assignments, find parameters θ^{new} that *maximize* the expected complete-data log-likelihood.

GMM: Expectation-Maximization (EM) Algorithm

Similar to K-means, by iterating between these steps we climb toward a local maximum of the log-likelihood, using soft assignments throughout.

EM for GMM

Set $\pi_k = 1/K$ and initialize $\{\mu_k, \Sigma_k\}$ for $k = 1, \ldots, K$
repeat
  E-step: Compute responsibilities (posterior probabilities)
    $\gamma_{ik} = p(z_i = k | x_i) = \frac{\pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}$ for all i, k
  M-step: Update parameters, for $k = 1, \ldots, K$,
    $N_k = \sum_{i=1}^{N} \gamma_{ik}$
    $\mu_k = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} x_i$
    $\Sigma_k = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^T$
    $\pi_k = \frac{N_k}{N}$
until stopping criterion is reached (e.g., log-likelihood change $< \epsilon$)
return centers μ_k, covariances Σ_k, mixture weights π_k, responsibilities γ_{ik}

Guarantees monotonic increase in log-likelihood at each iteration.

Initialization Strategies

Different ways to initialize the EM algorithm:

- **Random**: Select K data points randomly as μ_k, set $\Sigma_k = \text{Cov}(\mathbf{X})$
- **K-means**: Run K-means first, use resulting centers and within-cluster covariances
- **Multiple restarts**: Run EM several times with different random initializations, select solution with highest log-likelihood

Remark: In general, EM algorithms are a large class of algorithms which are commonly used to solve MLE or MAP estimation (see Exercise 13.3 for an application of EM to Bayesian linear regression) for problems involving latent random variables – see PRML Ch. 9 for details.

GMM: Derivation of M-Step Updates

The M-step updates are derived by maximizing the log-likelihood w.r.t. each parameter while holding the responsibilities γ_{ik} fixed.

Update for mean μ_k: Setting $\nabla_{\mu_k} \log p(\mathbf{X}|\theta) = 0$ yields:

$$\sum_{i=1}^{N} \gamma_{ik} \Sigma_k^{-1}(x_i - \mu_k) = 0 \implies \mu_k = \frac{\sum_i \gamma_{ik} x_i}{\sum_i \gamma_{ik}} = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} x_i.$$

We can interpret this as a weighted average of the x_i, with weight determined by the responsibility of the kth cluster in explaining x_i. The N_k is the effective number of points assigned to cluster k.

Update for covariance Σ_k: Similarly, maximizing with respect to Σ_k gives the weighted sample covariance (see Exercise 13.4):

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik}(x_i - \mu_k)(x_i - \mu_k)^T.$$

GMM: Derivation of M-Step Updates (Continued)

Update for mixing weights π_k: We maximize the log-likelihood with the constraint $\sum_k \pi_k = 1$ using a Lagrange multiplier λ. The Lagrangian is:

$$\mathcal{L}(\pi, \lambda) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \mathcal{N}_{ik} \right) + \lambda \left(\sum_{k=1}^{K} \pi_k - 1 \right),$$

where $\pi = (\pi_1, \ldots, \pi_K)$ and $\mathcal{N}_{ik} = \mathcal{N}(x_i; \mu_k, \Sigma_k)$.

Setting the derivative w.r.t. π_k to zero and solving (your exercise) gives $\lambda = -N$ and then:

$$\pi_k = \frac{\sum_i \gamma_{ik}}{\sum_i \sum_k \gamma_{ik}} = \frac{N_k}{N}.$$

In other words, the mixture weights are simply the proportion of the effective number of data points assigned to the kth cluster.

💡 We see that the update equations are interdependent, which necessitates the iterative EM approach.

EM vs. SGD for GMM Parameter Estimation

- The EM algorithm performs **coordinate ascent** on the log-likelihood:

$$L(\theta) = \sum_i \log \left(\sum_k \pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k) \right),$$

  alternating between the E-step and the M-step.
- **SGD-based training** instead directly maximizes $L(\theta)$ using gradient updates on minibatches:

$$\theta \leftarrow \theta + \eta \nabla_\theta L(\theta).$$

- **Comparison:**
  - EM uses closed-form updates, ensures monotonic likelihood increase.
  - SGD scales better to large or streaming datasets but may be slower and unstable, requiring learning rate tuning and regularization.
  - EM is typically faster for classical GMM clustering; SGD is useful when EM updates are intractable.

EM $\Rightarrow$ analytical, monotonic, but full-batch-based.

SGD $\Rightarrow$ scalable, flexible, but approximate.

Relationship to K-Means

K-means is a special case of GMM where assignments are "hard" instead of "soft." This can be seen by considering a GMM with fixed, shared, spherical covariances.

Let all covariance matrices be $\Sigma_k = \sigma^2 I$. The responsibility becomes:

$$\gamma_{ik} = \frac{\pi_k \exp\left(-\frac{1}{2\sigma^2}\|x_i - \mu_k\|^2\right)}{\sum_{j=1}^{K} \pi_j \exp\left(-\frac{1}{2\sigma^2}\|x_i - \mu_j\|^2\right)}$$

The limit as $\sigma^2 \to 0$ (assuming $\pi_k \neq 0$ for all k and $x_i \neq \mu_k$):

- The exponential term with the smallest squared distance $\|x_i - \mu_k\|^2$ will dominate all others and go to 1.
- All other terms will go to 0.
- The responsibility γ_{ik} becomes a 1-of-K ("hard") encoding:

$$\gamma_{ik} \to \begin{cases} 1 & \text{if } k = \arg\min_j \|x_i - \mu_j\|^2 \\ 0 & \text{otherwise} \end{cases}$$

- The M-step update for the mean μ_k becomes the K-means update: the average of all points assigned to that cluster.

GMM vs K-Means for Clustering

⚠ In a standard GMM, if a component $\mathcal{N}_k$ collapses onto a single data point, its covariance goes to zero and the likelihood can become infinite (see Experimental Exercise, part (g), for a solution to this issue). This is a potential issue not present in K-means.

Summary:

Property	**K-means**	**GMM**
Assignment	Hard (0/1)	Soft (probabilities)
Cluster shape	Spherical	Elliptical (any shape)
Cluster size	Similar	Can vary
Probabilistic	No	Yes
Complexity[6]	$O(NKdT)$	$O(NKd^2T)$
Speed	Fast	Slower
Uncertainty	No	Yes (via γ_{ik})

💡 Try the experimental exercises to better understand how these two clustering methods work.

[6] T = number of iterations. GMM with diagonal covariances has complexity $O(NKdT)$, same as K-means.

Exercise: Dissimilarity Measures for Standardized Data

Exercise 13

1. For centered data where each feature has zero mean: $\sum_{i=1}^{N} x_{ij} = 0$ for all j, define feature-wise standardized observations:

$$\tilde{x}_{ij} = \frac{x_{ij}}{\sigma_j}, \quad \sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N} x_{ij}^2$$

where σ_j is the standard deviation of feature j across all observations.

   (a) Show that $\frac{1}{N}\sum_{i=1}^{N} \tilde{x}_{ij}^2 = 1$ for all j.

   (b) Define sample correlation: $\rho(x_i, x_k) = \frac{1}{d}\sum_{j=1}^{d} \tilde{x}_{ij}\tilde{x}_{kj}$.
   Show that $\|\tilde{x}_i - \tilde{x}_k\|^2 = 2d(1 - \rho(x_i, x_k))$ when $\|\tilde{x}_i\|^2 = \|\tilde{x}_k\|^2 = d$, where $\|x\| := \|x\|_2 = (\sum_j x_j^2)^{1/2}$ denotes the Euclidean (ℓ_2) norm.

   (c) What does this imply for clustering based on correlation versus squared Euclidean distance?

   (d) Would a similar relationship between distance and correlation hold if we used the ℓ_1 distance instead of the ℓ_2 distance? Explain briefly why or why not.

Exercise: Convergence of K-Means

Exercise 13

2. Given samples $\{x_i\}_{i=1}^N$, $x_i \in \mathbb{R}^d$, consider the K-means clustering algorithm, which alternates between assigning points to the nearest cluster center and re-estimating the cluster means:

$$r_{ik} = \begin{cases} 1 & \text{if } k = \arg\min_j \|x_i - m_j\|^2, \\ 0 & \text{otherwise,} \end{cases} \qquad \text{and } m_k = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}.$$

   (a) Show that for a fixed assignment of the indicator variables $\{r_{ik}\}$, the objective function $J = \sum_{i=1}^N \sum_{k=1}^K r_{ik} \|x_i - m_k\|^2$ is minimized by choosing each cluster mean as the centroid of the points assigned to it, $m_k^* = \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}$.

   (b) Argue that, for a fixed set of centroids $\{m_k\}$, the reassignment step $r_{ik} = \mathbb{I}[k = \arg\min_j \|x_i - m_j\|^2]$, where $\mathbb{I}$ denotes the indicator function, always leads to a non-increasing value of J.

   (c) Using (a)-(b) and the facts that: (i) there are finitely many possible assignments for the discrete variables $\{r_{ik}\}$, (ii) each assignment has a unique optimum set of means $\{m_k\}$, prove that the K-means algorithm must converge after a finite number of iterations.

Exercise: EM for Bayesian Linear Regression

Exercise 13

3. Consider Bayesian linear regression with features $\phi_n \in \mathbb{R}^d$ and targets $y_n \in \mathbb{R}$, $n = 1, \ldots, N$. Assume Gaussian noise with precision β (i.e., variance $\sigma^2 = 1/\beta$) and prior $p(w \mid \alpha) = \mathcal{N}(0, \alpha^{-1} I_d)$.
   The E-step computes the posterior $q(w) := p(w \mid y, \alpha^{\text{old}}, \beta^{\text{old}}) = \mathcal{N}(m_N, S_N)$.
   The expected complete-data log likelihood is:

$$\begin{aligned} \mathbb{E}_q[\log p(y, w|\alpha, \beta)] =& \frac{d}{2} \log\left(\frac{\alpha}{2\pi}\right) - \frac{\alpha}{2}\mathbb{E}_q[w^T w] \\ &+ \frac{N}{2} \log\left(\frac{\beta}{2\pi}\right) - \frac{\beta}{2} \sum_{n=1}^{N} \mathbb{E}_q[(y_n - w^T \phi_n)^2]. \end{aligned}$$

   (a) For $w \sim \mathcal{N}(m_N, S_N)$, show that $\mathbb{E}_q[w^T w] = m_N^T m_N + \operatorname{tr}(S_N)$.
   (b) Show that maximizing with respect to α gives: $\alpha^{\text{new}} = \frac{d}{m_N^T m_N + \operatorname{tr}(S_N)}$.
   (c) Derive the analogous M-step re-estimation equation for β.

Exercise: Covariance M-Step for Gaussian Mixture

Exercise 13

4. In the EM algorithm for a Gaussian Mixture Model, the expected complete-data log-likelihood is given by (for $\gamma_{ik} \in [0,1]$):

$$Q(\{\Sigma_k\}) = -\tfrac{1}{2} \sum_{i=1}^{N} \sum_{k=1}^{K} \gamma_{ik} \left[\log |\Sigma_k| + (x_i - \mu_k)^\top \Sigma_k^{-1} (x_i - \mu_k) \right].$$

   (a) Let $\Sigma_k = \sigma_k^2 \in \mathbb{R}$ and $x_i, \mu_k \in \mathbb{R}$. Show that maximizing Q w.r.t. σ_k^2 gives $\sigma_k^{2,\text{new}} = \dfrac{1}{N_k} \displaystyle\sum_i \gamma_{ik} (x_i - \mu_k)^2$, where $N_k = \sum_i \gamma_{ik}$.

   (b) For $\Sigma_k \in \mathbb{R}^{d \times d}$, using the matrix identities $\nabla_A \log |A| = (A^{-1})^\top$ and $\nabla_A \operatorname{tr}(A^{-1}B) = -A^{-T} B A^{-T}$, set $\nabla_{\Sigma_k} Q = 0$ to derive

$$\Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top.$$

   (c) Interpret the role of γ_{ik} in the covariance update. What does γ_{ik} represent, and how does this differ from K-means where $r_{ik} \in \{0,1\}$?

Exercise: K-Means vs. GMM Clustering (Part 1/3)

Exercise 13 (Experimental)

Setup: For this entire exercise, you will use a single simulated dataset.

- **Generate Data:** Fix a random seed and create a data matrix $\mathbf{X} \in \mathbb{R}^{60\times 2}$ with $N = 60$ observations ($d = 2$ features) from three classes (20 per class).
- **Class Means:** Use the following mean vectors, with $\delta = 6$:
  $\mu_1 = (\delta, 0),\ \mu_2 = (0, \delta),\ \mu_3 = (-\delta, 0)$
- **Noise Covariance:** Use different covariance matrices for each class:

$$\Sigma_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad \text{(small, isotropic)}$$

$$\Sigma_2 = \begin{bmatrix} 4 & 0 \\ 0 & 4 \end{bmatrix} \quad \text{(large, isotropic)}$$

$$\Sigma_3 = \begin{bmatrix} 4 & 1.5 \\ 1.5 & 1 \end{bmatrix} \quad \text{(anisotropic/elliptical)}$$

- **Store:** Keep the data matrix $\mathbf{X}$ and the true class labels $y \in \{1, 2, 3\}^{60}$ fixed for all subsequent tasks.

Exercise: K-Means vs. GMM Clustering (Part 2/3)

Exercise 13 (Experimental)

Provide plots to visualize your results for the following exercises.

(a) **Baseline K-Means:** Run K-means clustering with $K = 3$ on the data $\mathbf{X}$. Qualitatively describe how well the cluster assignments match the true class labels. Check if there are any major mismatches.

(b) **Baseline GMM:** Fit a 3-component Gaussian Mixture Model (GMM) with full covariances to $\mathbf{X}$. Compare the hard cluster assignments (based on max responsibility) to the K-means results from (a). Do points near cluster boundaries get classified differently? Relate this to the spherical shape assumption of K-means versus the flexible, ellipsoidal shapes of GMM.

(c) **K-Means with Incorrect K:** Run K-means on $\mathbf{X}$ twice: once with $K = 2$ and once with $K = 4$. Describe which true classes are merged when $K = 2$ and which class is split when $K = 4$.

(d) **GMM with Incorrect K:** Repeat the previous step using a GMM. Fit it with $K = 2$ and $K = 4$. Does the GMM merge and split the same classes as K-means did?

Exercise: K-Means vs. GMM Clustering (Part 3/3)

Exercise 13 (Experimental)

(e) **K-Means on Scaled Data:** Standardize each feature in $\mathbf{X}$ to have zero mean and unit variance to obtain a new matrix $\tilde{\mathbf{X}}$. Run K-means with $K = 3$ on $\tilde{\mathbf{X}}$. How do these results compare to the unscaled results in (a)? Explain why scaling had this effect.

(f) **GMM on Scaled Data:** Now, fit a 3-component GMM to the standardized data $\tilde{\mathbf{X}}$. Compare the results to the unscaled GMM results in (b). Explain why the clusters with larger covariances in the original data may have dominated the density calculations.

(g) **GMM Failure Mode:**

- To demonstrate this, create a small problematic dataset: generate 15 points that lie approximately on a line by sampling $t \in [0, 10]$ and setting $(x, y) = (t, t + \epsilon)$ where $\epsilon \sim \mathcal{N}(0, 0.01)$.
- Fit a GMM with $K = 3$ components to this data. Inspect the eigenvalues of the fitted covariance matrices to identify near-singular matrices.
- Now, add λI (for a small λ) to each covariance matrix and refit the GMM. Compare the eigenvalues. Plot both the data and the results.